

Editor vi

- Pasa a modo inserción: **i I a A o O**
- Dentro de modo inserción: *tecleamos lo que queremos incluir*
- Pasa a modo comando/corrección/revisión: **<Esc>**
- Dentro de modo revisión:
 - borrar: **x X J dw dd**
 - moverse: **w ^F ^B ...**
 - deshacer: **u U**
 - recuperar buffer: **p P**
 - intercambiar: **xp ddp**
 - sale del editor guardando lo realizado: **:wq**
 - sale del editor sin guardar lo realizado: **:q!**
 - moverse entre líneas: **h j k l** *las cuatro flechas del teclado*

Make

- Por defecto (sin Makefile):

```
$make fi01
```

```
fi01: fi01.c
```

```
cc fi01.c -o fi01
```

```
cc fi01.c -o fi01 se ejecutará si fi01 tiene fecha  
anterior a la última modificación del fichero fi01.c
```

Make

- Para ejecutarlo:

```
$./fi01
```

Comando sed

- Selecciona línea/s de un fichero de entrada y el resultado se muestra en la salida estándar:

```
sed [-e] [dir1[,dir2][!]]funcion[args] nomFich
```

```
sed -f fichConf nomFich
```

```
sed -e funcion nomFich
```

```
sed -e funcion/arg1/arg2/arg3 nomFich
```

```
sed -e !funcion nomFich
```

```
sed -e dir1 funcion nomFich
```

```
sed -e dir1,dir2 funcion nomFich
```

Comando sed

- **sed** escribe por defecto en la salida estándar todas las líneas del fichero entrada (usando -n se puede evitar que esto pase)

funcion[args]:

- d** no selecciona línea (no la escribe)
- p** selecciona línea (la vuelve a escribir)
- s/arg1/arg2/** sustituye *arg1* por *arg2* (solo una vez por línea)
- s/arg1/arg2/g** sustituye *arg1* por *arg2* (siempre en cada línea)
- r arg1** inserta el fichero *arg1* después de cada línea seleccionada

Comando sed

<code>dir1 funcion</code>	la función se aplica a la línea <code>dir1</code>
<code>dir1, dir2 funcion</code>	la función se aplica a las líneas entre <code>dir1</code> y <code>dir2</code>
<code>funcion</code>	si no se pone dirección, la función se aplica a todas las líneas del fichero

```
$cd ~/tso/01/palabras
```

```
$sed -e '1d' dias
```

```
$sed -e '1,5d' dias
```

```
$sed -e 'd' dias
```

```
$sed -e '1p' dias
```

```
$sed -e '1,5p' dias
```

```
$sed -e 'p' dias
```

```
$cat dias
```

← dias no se modifica

Comando sed

`dir1 funcion/arg1/arg2/arg3` la función se aplica a la línea `dir1`

`dir1, dir2 funcion/arg1/arg2/arg3` la función se aplica a las líneas
entre `dir1` y `dir2`

`funcion/arg1/arg2/arg3` si no se pone dirección, la función se
aplica a todas las líneas del fichero

`$cd ~/tso/01/palabras`

`$sed -e '1s/lu/LU/' dias`

`$sed -e '1,3s/es/ES/' dias`

`$sed -e 's/es/ES/' dias`

`$sed -e 's/e/E/' dias`

`$sed -e 's/e/E/g' dias`

`$sed -e '3,5s/e/E/g' dias`

`$cat dias` *← dias no se modifica*

Comando sed

```
$cat valores
```

```
a1a1a1
```

```
b1b1b1
```

```
$cd ~/tso/01/palabras
```

```
$sed -e '1r valores' dias
```

```
$sed -e '6,7r valores' dias
```

```
$cat dias    ← dias no se modifica
```


Comando sed

- `dir1 dir2` pueden ser el símbolo `$` (significa final del fichero)

`$sed -e '$s/o/X/' dias` ← en la última línea sustituye o por X

`$sed -e '2,$s/o/X/g' dias` ← desde la segunda línea hasta el final
sustituye todas las letras o por X

- Las direcciones (`dir1 dir2`) y los argumentos (`arg1 arg2`)
pueden ser expresiones regulares

`$cd ~/tso/01/palabras`

`$sed -e '/u/d' dias` ← no escribe las líneas que tienen u

`$sed -e '/^m/d' dias` ← no escribe las líneas que empiezan por m

`$sed -e '/u/,/a/d' dias` ← no escribe entre u y a (repetidas veces)

`$sed -e '/u/,/a/s/es/ES/' dias` ← cambia es por ES entre u y a
(repetidas veces)

Comando sed

- La función se aplica al conjunto de líneas **complementario** al que se tomaría sin ese carácter !

```
dir1 !funcion
dir1, dir2 !funcion
!funcion
```

```
$cd ~/tso/01/palabras
```

```
$sed -e '1!d' dias      ← no escribe ninguna línea excepto la 1
                        (escribe solo la línea 1)
```

```
$sed -e '1,6!p' dias    ← reescribe solo la línea 7
                        (no reescribe desde la línea 1 a la 6)
```

```
$sed -e '/u/!d' dias    ← no escribe las líneas que no tienen u
                        (escribe solo las líneas que contienen u)
```

Comando sed

- Resumiendo, la estructura de **sed -e** es:

sed -e *parte_dirección acción argumentos* nomFich

sed [-e] [dir1[,dir2][!]]funcion[args] nomFich

sed -e *funcion* nomFich

sed -e *funcion/arg1/arg2/arg3* nomFich

sed -e *!funcion* nomFich

sed -e *dir1 funcion* nomFich

sed -e *dir1,dir2 funcion* nomFich

Comando sed

- **sed -f** aplica todas las órdenes del fichero de configuración a cada línea de texto del fichero de entrada

```
sed -f fichConf nomFich
```

```
$cat fConf
```

```
1,6d
```

```
7p
```

```
$cd ~/tso/01/palabras
```

```
$ sed -f fConf dias
```

```
domingo
```

```
domingo
```

```
$ sed -e '1,6d' -e '7p' dias
```

Sentencia for

- Se utilizar **for** en el intérprete de comandos (bash) para realizar tareas repetitivas

```
for variable in lista
```

```
do
```

```
    sent1
```

```
    ...
```

```
    sentn
```

```
done
```

```
for variable in lista ; do sent1 ; ... sentn ; done
```

Sentencia for

- En cada iteración *variable* toma un valor de *lista*
- *\$variable* referencia el contenido de *variable* en esa iteración

```
for variable in lista  
do  
    ...  
    $variable  
    ...  
done
```

Sentencia for

- Ejemplo *script* usando for

vi unFor

```
for indice in primer segun tercer
do
    echo ---principio iteracion
    echo $indice
    echo ---fin iteracion
done
```

\$bash unFor